

Jared Medlin

D209 – Data Mining I

Task 2

9-27-2024

Western Governors University

Part I – Research Question

A1. Question

Can customer churn be predicted by a decision tree?

I will be using a decision tree, of course.

A2. Objective and Goals

The goal of this analysis is to construct a decision tree model that will be able to predict customer churn with the given data set. This will help the business to determine how much customer churn there will be given certain data predictors.

Part II – Method Justification

B1. Explanation of Classification Method

Information sourced from scikit and IBM:

<https://scikit-learn.org/stable/modules/tree.html>

<https://www.ibm.com/topics/decision-trees>

I chose to use a decision tree because they are easy to work with and compatible with most of the data in this set.

Decision trees are effectively just a series of yes/no questions and then following a path based on the answers given, forming a tree shape. At the end of these paths there is a “leaf”. Each leaf outputs a binary answer of 0 or 1, in this case, will the customer churn or not. Excellent for a categorical variable with only 2 values.

B2. Assumption

The decision tree model assumes that the dataset is large enough to make meaningful predictions. Decision trees have a tendency to create very different trees with small variations, so small datasets will produce highly variant results.

B3. Justification of Packages

I will be using python due to its flexibility and extensive documentation.

NumPy is a standard for using math, which I will be using.

Pandas is critical for utilizing datasets.

Matplotlib is for visualizations/graphs.

Seaborn is also for visualizations/graphs.

Sklearn's packages are typically more specialized, in this case I'll be using it for the modelling.

Part III – Data Preparation

C1. Goal

I'll need to clean and transform the data for practical use in the decision tree model.

There's of course the initial checking for null values and duplicates, but I will also need to re-express the categorical variables as 1 and 0. Unlike task 1, I will not need to normalize the data.

C2. Variables

The target variable is 'Churn' which is categorical.

The other variables are:

Income – Numeric

Bandwidth_GB_Year – Numeric

Contacts – Numeric

Yearly_equip_failure – Numeric

InternetService – Categorical

Outage_sec_perweek – Numeric

Techie – Categorical

OnlineSecurity – Categorical

Port_modem – Categorical

StreamingMovies – Categorical

C3. Cleaning Steps

I'll only be cleaning the variables that I'm actually using, so variables like zip code will be left alone.

I'll start by checking for null values.

```
df.isnull().sum()  
#checking for null values and displaying as a sum
```

CaseOrder	0
Customer_id	0
Interaction	0
UID	0
City	0
State	0
County	0
Zip	0
Lat	0
Lng	0
Population	0
Area	0
TimeZone	0
Job	0
Children	0
Age	0
Income	0
Marital	0
Gender	0
Churn	0
Outage_sec_perweek	0
Email	0
Contacts	0
Yearly_equip_failure	0
Techie	0
Contract	0
Port_modem	0
Tablet	0
InternetService	2129
Phone	0
Multiple	0
OnlineSecurity	0
OnlineBackun	0

The only value that comes back with nulls is InternetService. But, just like in 208, this isn't actually true, it's just reading 'None' as null, so I'll need to fix that.

```
#InternetService doesn't actually have null values, "None" is a valid value  
df['InternetService'] = df['InternetService'].fillna('NoInternet')
```

Next I'll check for duplicates:

```
df.duplicated()  
#Duplicates check  
  
0      False  
1      False  
2      False  
3      False  
4      False  
...  
9995   False  
9996   False  
9997   False  
9998   False  
9999   False  
Length: 10000, dtype: bool
```

There are none.

Next I will create dummy variables of 1/0 for the binary categorical variables:

```
#Converting all of the categorical variables to dummy variables for the model  
#Variables to be converted into new dummy columns: Churn, InternetService, Techie, OnlineSecurity, Port_modem, StreamingMovies, StreamingTV  
df['DummyChurn'] = [1 if v == 'Yes' else 0 for v in df['Churn']]  
df['DummyTechie'] = [1 if v == 'Yes' else 0 for v in df['Techie']]  
df['DummyOnlineSecurity'] = [1 if v == 'Yes' else 0 for v in df['OnlineSecurity']]  
df['DummyPort_modem'] = [1 if v == 'Yes' else 0 for v in df['Port_modem']]  
df['DummyStreamingMovies'] = [1 if v == 'Yes' else 0 for v in df['StreamingMovies']]  
df['DummyStreamingTV'] = [1 if v == 'Yes' else 0 for v in df['StreamingTV']]
```

For the categorical variable InternetService, there are 3 values, so I'll use one-hot encoding to make each value its own binary variable:

```
#One hot encoding for InternetService  
df = pd.get_dummies(data=df, columns=['InternetService'], dtype=int)  
list(df.columns)
```

Then I'll create a new dataframe with only the relevant variables:

```
#Creating a new dataframe with only relevant columns
df2 = df[['Outage_sec_perweek', 'Income', 'Contacts', 'Yearly_eq
df2.describe()
```

	Outage_sec_perweek	Income	Contacts	Yearly equip.
count	10000.000000	10000.000000	10000.000000	10000
mean	10.001848	39806.926771	0.994200	0
std	2.976019	28199.916702	0.988466	0
min	0.099747	348.670000	0.000000	0
25%	8.018214	19224.717500	0.000000	0
50%	10.018560	33170.605000	1.000000	0
75%	11.969485	53246.170000	2.000000	1
max	21.207230	258900.700000	7.000000	6

Data is now ready to be used.

See attached .csv for full set of columns used.

C4. Cleaned Data Set

SEE ATTACHED FILE

Part IV – Analysis

D1. Splitting Data

```
#splitting datasets for X and y:
xdf = df2.drop(['DummyChurn'], axis=1).copy()
ydf = df2['DummyChurn'].copy()
```

```
#Creating training/test sets
xtrain, xtest, ytrain, ytest = train_test_split(xdf, ydf, train_size = 0.8, test_size = 0.2, random_state = 33)
```

```
#Creating .csv files
xtrain.to_csv('C:/Users/weepo/Documents/WGU\D209 Task 2/Finished Data/xtrainTASK2.csv', index=False)
xtest.to_csv('C:/Users/weepo/Documents/WGU\D209 Task 2/Finished Data/xtestTASK2.csv', index=False)
ytrain.to_csv('C:/Users/weepo/Documents/WGU\D209 Task 2/Finished Data/ytrainTASK2.csv', index=False)
ytest.to_csv('C:/Users/weepo/Documents/WGU\D209 Task 2/Finished Data/ytestTASK2.csv', index=False)
```

SEE ATTACHED .CSV FILES

D2. Description of Analysis Technique

Code sourced from: <https://scikit-learn.org/stable/modules/tree.html> and its various subpages.

I'll be using DecisionTreeClassifier. When it comes to the depth of the tree (how many levels down each branch goes). Too many branches can cause issues, but too few can as well. I will be using my handy GridSearch to determine the optimal number (under 8, let's not go crazy here).

```
#Max Depth
#Code from scikit https://scikit-learn.org/stable/modules/tree.html
#And https://scikit-learn.org/dev/modules/generated/sklearn.model_selection.GridSearchCV.html
params_dt = {'max_depth' : [1, 2, 3, 4, 5, 6, 7, 8]}

#Initial Decision Tree
dt = DecisionTreeClassifier(random_state = 33)

#GridSearch
dtgrid = GridSearchCV(estimator=dt, param_grid=params_dt, scoring='roc_auc', cv=5, n_jobs=-1)

#Gridfit to training data
dtgrid.fit(xtrain, ytrain)

#best fit
bestfit = dtgrid.best_estimator_

#Predict
ypredict = bestfit.predict(xtest)

#Accuracy report
accuracytest = accuracy_score(ytest, ypredict)
print('Accuracy of best decision tree: {:.2f}'.format(accuracytest))

Accuracy of best decision tree: 0.83
```

```
bestfit
```

```
DecisionTreeClassifier
DecisionTreeClassifier(max_depth=5, random_state=33)
```

And there you have it, max depth will be 5.

All that's left to do is create the tree with a depth of 5 and make some accuracy, AUC, and error metrics to measure the effectiveness of the model:

```
#Creating Tree
dt2 = DecisionTreeClassifier(max_depth=5, random_state=33)
dt2.fit(xtrain, ytrain)

#Predictions are in
dt2predict = dt2.predict(xtest)

#Accuracy Test
dt2acc = accuracy_score(ytest, dt2predict)
print('Test set accuracy of Decision Tree is: {:.2f}'.format(dt2acc))

#Probabilities
dt2prob = dt2.predict_proba(xtest)[: ,1]

#ROC and AUC
dt2auc = roc_auc_score(ytest, dt2prob)

#MSE Error
mse = mean_squared_error(ytest, dt2prob)
print(f'The Mean-squared Error is: {mse}')

#Classification Report
print(classification_report(ytest, dt2predict))

print('The AUC score is: {:.2f}'.format(dt2auc))
```

Test set accuracy of Decision Tree is: 0.83
The Mean-squared Error is: 0.11375256709521167

	precision	recall	f1-score	support
0	0.83	0.95	0.89	1447
1	0.80	0.50	0.61	553
accuracy			0.83	2000
macro avg	0.82	0.73	0.75	2000
weighted avg	0.82	0.83	0.81	2000

The AUC score is: 0.90

D3. Code

SEE CODE ATTACHED

Part V – Summary and Implications

E1. Explanation

The accuracy of the model was 0.83. This means that, for every prediction, the model was correct 83% of the time. That's a pretty good value, which means the model is likely working as intended. The Mean-squared error is 0.1138. This is the average of the square of the errors in the model. This means that the magnitude of the errors present in this model was fairly low. 0.1138 is a decently low value.

E2. Results and Implications

The model is decent. It's predictive ability is clearly higher than random, so it does have some value there. There are not too many conclusions that can be drawn from this, however, as decision trees have some drawbacks and the dataset is a static piece of data. I can say this: given a random customer in this dataset, I am about 83% confident that this model would be able to predict whether the customer will churn.

E3. Limitations

The drawback of this model is similar to the one in task 1, in that it is limited to its small, static dataset, and thus not very applicable outside of that. The decision tree, in particular, is bad for extrapolating outside the dataset, as decision trees have a tendency to overfit and not translate well to new data.

E4. Course of Action

I cannot recommend that this model be used for any practical use outside of this data set. It can be used to analyze this set in particular as a sort of fun project, but outside of this set it is likely useless and will be outclassed by several different models given the same training.

I would recommend investigating other models to predict churn, if that is the goal.

Part VI: Demonstration

F. Panopto Video

SEE ATTACHED LINK

G. Sources

IBM. (n.d.). *What are decision trees?*. Retrieved September 27, 2024, from <https://www.ibm.com/topics/decision-trees> IBM. (n.d.). *What is k-nearest neighbors (k-NN)?* IBM. <https://www.ibm.com/topics/knn>

Scikit-learn. (n.d.). *sklearn.tree.DecisionTreeClassifier*. Scikit-learn documentation. Retrieved September 27, 2024, from <https://scikit-learn.org/dev/modules/generated/sklearn.tree.DecisionTreeClassifier.html>

H. Sources Cited

Sources cited in text or code where applicable.

I. Demonstrate Professionalism

Thank you for reading!